

REHARNESS: 基于 AST 的 C 设备驱动形式化寄存器交互规约提取

Anonymous Author(s)

Abstract

设备驱动占据了操作系统代码的大部分，但其寄存器级行为在很大程度上仍然是隐式的——通过临时宏、指针运算和框架相关的惯用法埋藏于 C 源码中。我们提出了 REHARNESS，一个使用 libclang AST 分析结合流敏感数据流和污点追踪从 C 驱动中提取形式化寄存器交互序列 (RIS) 的流水线，并辅以 LLVM-IR 证据来恢复隐藏在头文件内联辅助函数中的 MMIO 操作。超越原始 MMIO，RIS 对有类型的 regmap/I2C/MFD 事务和功能状态操作 (值绑定、驱动状态更新和输出写入) 进行建模，捕获软件缓冲区与设备寄存器之间的数据流。提取的 RIS 为后端无关的函数/设备规约和可插拔的多后端生成器提供输入，后者通过提示驱动的 LLM 后端生成用户空间测试工具 (harness)、裸金属 C、Linux 内核模块和 Rust no_std 驱动；每个候选方案都通过独立的编译、AST 和追踪验证来把关，而非直接信任其输出。评估工件由版本固定的、机器可读的流水线生成，而非手工转录的表格。在 19 个 Linux 驱动上，REHARNESS 提取了 485 个操作：366 个符号地址、64 个固定地址和 41 个计算地址访问，包括 73 个读-改-写模式和 123 个条件。我们额外评估了 3 个真实多源 Linux 模块，包含 19 个翻译单元和 27447 行代码。在冻结的确定性发射器基线下，测试工具和裸金属输出在 19/19 个单源用例中编译通过，Linux 输出在 18/19 个用例中编译通过；所有三个后端均成功编译每个多源模块。严格的语义就绪性被刻意设定得更加保守 (Linux 为 2/19)，因为未知的变换、不安全的动态地址和未绑定的子系统状态被视为阻碍因素而非被静默近似。确定性生成的驱动额外通过了 edu PCI 设备上的值级 QEMU 验证和真实 gpio-ftgpio010 驱动上的偏移/顺序追踪验证，并且 LLM 生成的多源 DesignWare APB SSI 控制器四后端输出通过了全部 18 项独立语义检查。

CCS Concepts: • Computer systems organization → Embedded systems; • Software and its engineering → Software development techniques.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASPLOS '27, 2027,

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/XX/XX

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Keywords: 设备驱动, 形式化规约, 程序分析, 代码生成, LLM 辅助综合, Rust

ACM Reference Format:

Anonymous Author(s). 2026. REHARNESS: 基于 AST 的 C 设备驱动形式化寄存器交互规约提取. In *Proceedings of Proceedings of the 52nd Annual International Symposium on Computer Architecture (ASPLOS '27)*. ACM, New York, NY, USA, ?? pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 引言

设备驱动是操作系统内核中数量最多且最易出错的组件 [??]. 一个 Linux 版本就包含数万个驱动源文件，每个文件通过内存映射 I/O (MMIO) 读写、中断处理程序和框架回调注册来编码复杂的硬件协议。尽管对驱动可靠性已有数十年的研究 [???], 驱动的寄存器级行为在很大程度上仍然是隐式的——通过宏、指针运算和临时数据结构表达在 C 源码中，难以进行自动化推理。

核心挑战是提取：将原始 C 源码转化为机器可读的规约，描述驱动对硬件寄存器做了什么、以什么顺序、在什么条件下。这一规约是验证 [?], 测试 [?], 向安全语言翻译 [?] 以及 AI 辅助的驱动综合 [?] 的基础。

然而，现有的提取工具运作在浅层。它们使用正则表达式来匹配 MMIO 函数调用 (如 readl, writel)，但在四种常见模式上会失败：(1) 通过 #define 宏定义的寄存器偏移在正则匹配下解析为零，(2) 隐藏在从驱动入口点调用的包装函数内部的 MMIO 访问，(3) 控制寄存器访问路径的分支条件，(4) 涉及基址指针和偏移的算术组合的地址表达式。

我们提出了 REHARNESS，一个通过 AST 级分析来解决这些限制的流水线。REHARNESS 使用 libclang 在完整预处理上下文中解析 C 翻译单元，使其能够解析宏展开后的寄存器偏移 (例如 GPIO_INT_EN → 0x20)，这些偏移对正则工具是不可见的。它执行过程间调用图分析，通过受控深度的包装函数内联来暴露隐藏在辅助函数中的 MMIO 操作。它使用谓词栈跟踪分支条件，区分 then 路径和取反的 else 路径，将守卫条件附加到每个寄存器操作，并为分支相关的 RMW 更新构造条件表达式。关键的是，它执行流敏感数据流和污点追踪：它将 ioremap 调用识别为 MMIO 基址指针来源，将 readl 返回值追踪为读取污点数据，并检测读-改-写 (RMW) 模式，即读取值被修改后写回同一地址。当 AST 级分析存在盲点——即内核头文件中定义的 static inline MMIO 辅助函数——时，REHARNESS 将翻译单元编译为 -O1 下的 LLVM IR，并从 IR 中恢复内联的 MMIO 操作作为补充证据，按函数和偏移将它们与 AST 提取结果合并。

REHARNESS 的输出是一种形式化寄存器交互序列 (RIS) 规约语言，设计为既可人类阅读又可机器解析。每个 RIS

模块记录一个驱动函数的寄存器操作序列，并标注符号寄存器名称、分支条件和语义意图。RIS 不限于 MMIO：有类型的事务操作对 regmap、I2C/SMBus 和公共 MFD 辅助函数访问进行建模，而不会将其句柄与内存地址混淆；功能状态操作——值绑定、驱动状态更新和输出写入——捕获数据在软件缓冲区与设备寄存器之间的流动方式。功能状态层使得生成的 SPI 驱动能够保持缓冲区推进相对于 FIFO 寄存器访问的精确源码顺序，这是功能（而非仅仅是寄存器级）等价性的必要条件。

在原始提取之上，REHARNESS 执行语义推断：它将 RIS 模块提升为后端无关的函数规约（FunctionSpec），捕获函数的角色（例如 interrupt_ack）、执行上下文、状态绑定、效果以及 Hoare 风格的前置和后置条件。函数规约组合为设备级规约（DeviceSpec），对设备的状态、资源、寄存器映射和不变式进行建模。这些形式化规约可与后端特定的绑定文件（.bind）配对，为多个目标生成驱动代码：用于测试的用户空间测试工具、用于嵌入式系统的裸金属 C、以及 Linux 内核模块。

最后，REHARNESS 通过可插拔的多后端架构生成目标代码。每个后端是一个自包含的插件——一个包含生成模块和自然语言提示模板的目录——在导入时由注册表发现。提供了四个后端：用户空间测试工具 C、裸金属 C、Linux 内核模块和 Rust no_std（使用类型安全的 tock-registers MMIO 抽象而非原始指针解引用）。代码发射委托给一个 LLM，它接收证据 JSON（RIS 操作、寄存器映射、绑定和源码事实）以及后端的翻译规则提示；LLM 永远不会看到原始 C 源码。由于 LLM 输出不可信任，每个候选方案都通过独立的验证循环来把关：带严格警告的编译、针对操作合约的生成代码 AST 检查、以及与 RIS 导出的预言机进行追踪比较，结构化的失败反馈驱动有界的修复迭代。

本文的贡献如下：

- 一种形式化寄存器交互序列（RIS）语言，捕获带有符号寄存器解析、分支条件和 RMW 检测的 MMIO 操作；有类型的 regmap/I2C/MFD 事务；以及保持缓冲区到寄存器数据流的功能状态操作（值绑定、状态更新、输出写入）——作为驱动行为提取的规范中间表示。
- 一个混合提取流水线：以基于 libclang 的 AST 分析和流敏感数据流及污点追踪作为主路径，辅以 LLVM-IR 证据来恢复隐藏在头文件内联 static inline 辅助函数中的 MMIO 操作。
- 从 RIS 模块到函数级和设备级形式化规约的后端无关语义推断，包括回调角色推断、状态绑定和效果建模。
- 一种可插拔的多后端生成架构，其中每个后端是一个提示加模块的插件；四个后端（测试工具 C、裸金属 C、Linux 内核模块、带 tock-registers 的 Rust no_std）共享一个证据合约，并且所有 LLM 发射的候选方案都通过独立的编译、AST 和追踪验证来把关。
- 在 19 个 Linux 驱动上的可复现评估，生成的表格与机器可读的结果关联，在冻结的确定性基线

下，测试工具、裸金属和 Linux 后端的编译通过率分别为 19/19、19/19 和 18/19，PCI 和平台设备上的确定性 QEMU 验证，以及 LLM 生成的四后端 DesignWare APB SSI 输出的 18/18 独立语义检查结果。

2 动机

2.1 正则提取的局限性

为了说明 REHARNESS 的动机，我们考察现有实践中正则提取工具 driver-harness [?] 的失败模式。它的方法是用正则表达式扫描 C 源码文本来检测 MMIO 调用。虽然轻量，但这种策略在真实驱动代码中普遍存在的四种模式上系统性失败。

模式 1：宏定义的寄存器偏移。 Linux 驱动通过预处理器宏定义寄存器布局：

```
#define GPIO_INT_EN    0x20
#define GPIO_INT_CLR   0x30
writel(0, base + GPIO_INT_EN);
```

匹配 writel(..., ... + ...) 的正则可以捕获调用点，但无法将 GPIO_INT_EN 解析为其数值 0x20。基线工具默认将所有此类访问报告为偏移 0，丢失了区分不同寄存器的关键信息。在我们的评估中，这导致 GPIO 和 virtio-MMIO 驱动中 100% 的寄存器访问被错误报告为位于偏移 0。

模式 2：包装函数。 驱动通常将 MMIO 原语包装在辅助函数中：

```
static void gpio_setbit(u32 val, void
__iomem *base, u32 off) {
    u32 r = readl(base + off);
    writel(r | val, base + off);
}
```

对主回调函数执行的正则会遗漏 gpio_setbit 及类似包装函数内部的所有 MMIO 操作。没有过程间分析，驱动整个寄存器访问足迹就是不完整的。

模式 3：条件寄存器访问。 寄存器序列通常取决于运行时条件：

```
if (type & IRQ_TYPE_EDGE_BOTH) {
    writel(val | BIT(line), base +
        GPIO_INT_BOTH_EDGE);
}
```

基线工具记录了 writel 操作但丢弃了守卫条件，丢失了该写入仅对边沿触发中断发生的语义上下文。

模式 4：算术地址计算。 基址指针通常通过成员访问链计算：

```
struct ftggpio_gpio *g = gpiochip_get_data(gc
);
readl(g->base + GPIO_INT_STAT);
```




Figure 1. REHARNESS 的实现架构，展示仓库组件和从 C 源码经过形式化 RIS 到 prompt 驱动后端插件的产物流，所有生成的候选都经过独立验证门控。

解析 $g \rightarrow \text{base}$ 需要追踪初始化它的 `ioremap` 调用，而这又需要流敏感的污点传播——远超正则表达式的能力。

2.2 设计目标

REHARNESS 的设计旨在克服这些限制，同时保持对下游工具链至关重要的三个属性：

1. **精确性：**每个寄存器访问必须保留在其最精确的可靠地址类别中：静态可推导时使用符号名称和偏移，否则使用显式的固定地址或运行时计算地址，而非伪造的符号。
2. **完备性：**不应静默丢弃从驱动入口点可达的任何 MMIO 操作。包装函数内联和过程间分析必须覆盖到有界深度的完整调用图。
3. **形式语义：**输出必须是形式化规约语言，而非临时 JSON 格式，使其可直接用作验证、测试和代码生成工具的输入。

3 系统概述

REHARNESS 组织为包含四个主要阶段的流水线，如图 ?? 所示。

阶段 1: RIS 提取。提取阶段实现在 `src/extractor/tu.py`、`src/extractor/macros.py` 和 `src/extractor/ast_model.py`。该阶段使用 `libclang` 解析 C 源码，保留预处理上下文，并提取 `#define` 偏移量的宏表。`.c` 和 `.h` 输入均被接受；头文件中定义的 `static inline` 函数被内联到提取缓存中，从而使头文件辅助函数中的寄存器访问被归因于其调用者。目标函数和回调初始化绑定从 AST 中发现。随后，提取器构建基于 `libclang` 的调用图，并执行带有保护谓词栈的流敏感数据流分析。除了 MMIO 读、写和 RMW 操作外，数据流分析还记录功能状态操作：`ValueBind`（从软件缓冲区绑定的变量）、`StateWrite`（对驱动状态的更新，如缓冲区游标或剩余长度计数器）以及 `OutputWrite`（从寄存器读取后存回软件缓冲区的存储操作）。一个互补的 IR 分析（`src/extractor/ir_enhance.py`）使用 `clang -O1 -emit-llvm` 编译翻译单元，从内联 IR 中恢复 MMIO 操作（其中 Linux 的 `readl/writel` 变为带有调试信息内联链的可识别内联汇编模式），并将 AST 结果中缺失的（函数、偏移）对作为补充证据进行合并。最终输出是一个形式化的 RIS 规约，以符号化寄存器名称、分支守卫、宽度标注和逐操作源码证据记录每个操作。

阶段 2: 语义推断。语义推断实现在 `src/extractor/spec_ir` 和 `src/extractor/spec.py`。该阶段用后端无关的语义来丰富原始的 RIS 模块：从回调表字段和函数名提示中推断函数角色，将 C 参数和返回类型抽象为形式化类型（例如，`struct irq_data *  $\rightarrow$  LogicalIRQ`），将诸如 $g \rightarrow \text{base}$ 的 MMIO 基地址表达式绑定到设备状态，并根据角色语义推导效果和 Hoare 风格的契约。最终结果为

每个驱动函数生成一个 `FunctionSpec`，并组合生成一个包含状态、资源、寄存器映射和不变式的 `DeviceSpec`。

阶段 3: 后端插件。代码生成组织为插件架构（`src/generator/register`）。每个后端占据一个目录，其中包含一个 Python 模块（声明其名称、输出语言和绑定构造方式）和一个 `prompt.md` 翻译规则模板；注册表在导入时自动发现插件，因此添加目标只需放入一个新目录即可。系统提供四个后端：具有模拟 MMIO 内存和跟踪日志的用户空间 `harness`、可移植的裸机 C、带有平台/PCI/GPIO/IRQ/时钟提供者粘合代码的 Linux 内核模块，以及使用 `tock-registers crate` 实现类型安全 MMIO 的 Rust `no_std` 驱动（通过 `register_structs!` 声明寄存器映射，构造函数之外不存在原始指针解引用）。后端的 `make_bind` 步骤保持确定性；代码发射委托给下文描述的 LLM 桥。

阶段 4: LLM 发射与验证门控。LLM 桥（`src/generator/llm_bridge`）将形式化制品归约为一份证据 JSON——驱动身份、设备类别、寄存器映射、逐模块操作列表、原语/类型/状态/回调绑定，以及选定的源码事实——并用其实例化后端的提示模板。LLM 仅看到此证据契约，绝不接触原始 C 源码，因此生成的代码受限于已提取的语义。由于 LLM 输出不可信，每个候选都会通过一个独立的验证门控：严格警告级别的编译、对生成的代码进行 AST 检查以确保每个发射的寄存器操作都携带有效的降低收据（恰好一次、正确的类型、宽度和字节序），以及与 RIS 派生的预言机进行跟踪比较。结构化的失败反馈驱动有界修复迭代（`src/synthesis.py`）；仅当所有门控通过时，候选才被接受。

4 形式化 RIS 语言

REHARNESS 的核心输出是 RIS（寄存器交互序列）规约语言。RIS 设计为既可读又可机器解析，具有支持真实驱动中全部 MMIO 操作词汇的形式化语法。

4.1 语法

```
driver <name> v<version> {  
  module <function_name> {  
    <var> := R(<width>, <addr>) [-- <intent  
      >]  
    W(<width>, <addr>) = <expr> [-- <intent  
      >]  
    RMW(<width>, <addr>) = <transform> [-- <  
      intent>]  
    TXN_R/TXN_W/TXN_U(<target>, <selector>)  
      [-- <intent>]  
    <var> := VALUE(<expr>)  
    STATE(<lvalue>) := <expr>  
    OUT(<target>) := <expr>  
    IF <guard> { <ops> } [ELSE { <ops> }]  
    LOOP <count> { <ops> }  
    DELAY(<cycles>)  
  }  
}
```

每个 module 对应一个驱动函数（框架回调或辅助函数）。模块内的操作按源码中的出现顺序排列，保留程序的执行序列。

4.2 操作

读 (R)：带有绑定变量供后续使用的寄存器读取操作。宽度为 B1、B2、B4、B8 之一（由 MMIO 调用的后缀推断：readl → B4, readw → B2, 以此类推）。地址被解析为以下三种形式之一：

- **符号化**：g->base.GPIO_INT_EN——已知基址表达式加上已解析的寄存器宏。
- **固定**：0x20——无宏解析的字面偏移量。
- **计算型**：[g->base + idx]——无法静态解析的动态地址表达式。

写 (W)：带有表达式值的寄存器写入操作，该值可以是常量、变量或 Expr 代数中的复合表达式。

读-改-写 (RMW)：当 writel 的值是对同一地址的 readl 返回值进行修改的结果时检测到。这是中断掩码/解掩码操作的关键模式，其中必须在不影响其他位的情况下切换单个位。

条件 (IF)：从 if/else 语句中提取分支条件，并将其附加到各分支内的操作上。谓词栈保留嵌套的守卫条件，并区分 then 路径与取反的 else 路径。

循环 (LOOP)：从 for/while 循环中提取的迭代构造，在可静态推导时附带计数表达式。

延迟 (DELAY)：时序操作 (mdelay、udelay、msleep) 转换为纳秒等效计数。

事务 (TXN_R/TXN_W/TXN_U)：用于 regmap、I2C/SMBus 和公共 MFD 辅助函数访问的类型化操作。目标句柄和寄存器/命令选择器是独立的表达式，永远不会被折叠为 MMIO 地址；选择器常量记录在单独的事务映射中。TXN_U 表示带有显式掩码、值和掩码替换语义的掩码更新。识别由来源门控（由 libclang 声明位置证明的公共、类型定义的 API），而非逐驱动名称的白名单。

功能状态操作 (VALUE/STATE/OUT)：捕获软件缓冲区与设备寄存器之间数据流的操作。VALUE 从缓冲区加载中绑定变量（例如 txw := VALUE*(u8*)(dws->tx)）；STATE 记录对驱动状态的更新，如缓冲区游标或剩余长度计数器（例如 STATE(dws->tx) := dws->tx + dws->n_bytes）；OUT 记录从寄存器读取后存回软件缓冲区的存储操作（例如 OUT*(u8*)(dws->rx) := rxw）。这些操作保留了缓冲区移动相对于 FIFO 寄存器访问的源码顺序。对于 SPI 发送路径，规范序列 VALUE(txw) → STATE(tx += n_bytes) → W(DR, txw) → STATE(tx_len--) 必须在翻译中保持不变，才能使生成的驱动真正传输数据，而不仅仅是触碰相同的寄存器。

4.3 表达式代数

RIS 中的值和守卫条件用代数表达式语言表示：

Expr ::= Const(n) | Var(x) | BinOp(op, left, right) | Ite(guard, then, else) | Bits(hi, lo, expr) | Top

BinOp 支持算术、位运算、关系和逻辑运算符。Ite 表示路径条件值，由所有后端以 C 条件表达式形式发射。Top 哨兵值表示无法静态解析的值。

表达式示例：

```
0x1 << irqd_to_hwirq(d)
-> BinOp{Shl, Const(1), Var("irqd_to_hwirq
(d)")}
0x0 ^ 0xffffffff
-> BinOp{BitXor, Const(0), Const(0
xffffffff)}
```

4.4 示例

Listing ?? 展示了提取的 gpio-ftgpio010 驱动，即 Faraday FTGPIO010 GPIO 控制器。每个模块对应一个通过 Linux irq_chip 和 gpio_chip 框架表注册的回调。

Listing 1. gpio-ftgpio010 的提取 RIS 规约节选（14 个模块，32 个操作，包括通用库摘要）

```
driver gpio-ftgpio010 v0.1.0 {
  module ftgpio_gpio_ack_irq {
    W(B4, g->base.GPIO_INT_CLR) = (0x1 <<
      irqd_to_hwirq(d))
    -- Interrupt
  }
  module ftgpio_gpio_mask_irq {
    val := R(B4, g->base.GPIO_INT_EN) --
      Interrupt
    RMW(B4, g->base.GPIO_INT_EN) = val --
      Interrupt
  }
  module ftgpio_gpio_unmask_irq {
    val := R(B4, g->base.GPIO_INT_EN) --
      Interrupt
    RMW(B4, g->base.GPIO_INT_EN) = val --
      Interrupt
  }
  module ftgpio_gpio_set_irq_type {
    reg_type := R(B4, g->base.GPIO_INT_TYPE)
    -- Interrupt
    reg_level := R(B4, g->base.
      GPIO_INT_LEVEL) -- Interrupt
    reg_both := R(B4, g->base.
      GPIO_INT_BOTH_EDGE) -- Interrupt
    RMW(B4, g->base.GPIO_INT_TYPE) =
      reg_type
    RMW(B4, g->base.GPIO_INT_LEVEL) =
      reg_level
    RMW(B4, g->base.GPIO_INT_BOTH_EDGE) =
      reg_both
  }
  module ftgpio_gpio_irq_handler {
    stat := R(B4, g->base.GPIO_INT_STAT_RAW)
    -- Interrupt
  }
  module ftgpio_gpio_set_config {
    val := R(B4, g->base.
      GPIO_DEBOUNCE_PRESCALE) -- Config
    IF (val == deb_div) {
```

```

441     val := R(B4, g->base.GPIO_DEBOUNCE_EN)
442     -- Config
443     RMW(B4, g->base.GPIO_DEBOUNCE_EN) =
444         val -- Config
445 }
446 ...
447 }
448 module ftdpio_gpio_probe {
449     W(B4, g->base.GPIO_INT_EN) = 0x0 --
450         Init
451     W(B4, g->base.GPIO_INT_MASK) = 0x0 --
452         Init
453     ...
454 }
455 }

```

全部 22 个直接源码寄存器操作加上从类型化通用 GPIO 配置中恢复的 10 个操作均被解析为符号化地址 (100% 符号化率)。掩码/解掩码回调正确地呈现了 RMW 模式 (读取当前中断使能掩码, 修改后写回)。set_config 模块在去抖使能 RMW 周围保留了条件守卫 (val == deb_div)。

5 基于 AST 分析 的 RIS 提取

5.1 翻译单元解析

REHARNES 使用 libclang 将每个 C 源文件解析为未保存的翻译单元 (unsaved translation unit), 从而保留宏展开记录。解析器以容错模式运行, 能够跳过常规编译器会中止的语法错误 (这对于依赖分析环境中不可用的内核头文件的驱动程序至关重要)。解析生成完整的 AST 游标树以及基于预处理记录构建的 MacroTable。

MacroTable 将翻译单元中的每个 #define 映射到其展开值。对于整数值的寄存器宏 (例如 #define GPIO_INT_EN 0x20), 我们记录数值偏移量。该表是实现符号地址解析的关键数据结构: 当数据流分析器在地址表达式中遇到对 GPIO_INT_EN 的引用时, 可将其解析为 0x20, 并与基址指针配对以生成 Symbolic 地址。

5.2 函数识别与回调检测

目标函数通过遍历 AST 中源位置在目标文件内的函数定义来识别。每个函数被建模为一个 Func 对象, 包含其名称、参数列表、返回类型、游标、源路径和链接标识。外部可见的定义保留其链接器符号, 而文件局部函数使用源限定的标识。因此, 两个翻译单元可以同时定义 static helper, 而不会拒绝模块或将一个 helper 的 MMIO 摘要传播到另一个中。

回调条目从指定初始化和成员赋值中恢复, 同时追踪所属的结构体类型。字段和结构体与角色表 (FIELD_ROLE) 进行匹配; 例如, irq_ack = ftdpio_gpio_ack_irq 成为完整绑定 irq_chip.irq_ack, 角色为 interrupt_ack。没有识别到框架绑定的函数被分类为 helper, 并可能被内联到调用者中。

5.3 流敏感数据流与污点追踪

提取的核心是逐函数的流敏感数据流分析。对于每个函数, 我们按源代码顺序遍历其调用点, 维护一个抽象存储, 将变量名映射到取自有限域的抽象值 (AbsVal):

```

AbsVal ::= BasePtr(b) | Offset(b, n, r) | ReadTaint(a, n) |
          Const(n) | SymExpr(e) | Top

```

污点源: ioremap 及其直接或 devres 管理的变体被识别为 MMIO 基址指针源。接收映射的变量绑定到 BasePtr。

地址解析: 当遇到 MMIO 读/写调用时, 其地址参数根据抽象存储和宏表进行符号化求值。该求值通过将指针变量的 BasePtr 与宏表中的 Offset 值结合来解析 base + REG 表达式, 生成携带设备上下文和寄存器名称的 Symbolic 地址。

读污点: readl 调用的返回值绑定到 ReadTaint(addr), 记录读取的地址。当后续的 writel 使用一个求值为相同地址的 ReadTaint 的值时, REHARNES 检测到读-改-写 (read-modify-write) 模式, 并发出 RMW 操作而非分离的读和写操作。

表达式求值: 递归下降求值器处理抽象值上的算术 (+, -, <<, >>), 位运算 (&, ^) 和一元 (~) 操作。当两个操作数均为 Const 时应用常量折叠; 否则, 表达式被捕获为 SymExpr 或 BinOp 用于形式化 RIS 输出。

5.4 包装函数内联

当调用点引用了一个已分析的、非框架原语的函数时 (不在 FRAMEWORK_FNS 中), REHARNES 检查该函数是否已被先前提取。如果是, 其操作在调用点实例化: 形式参数在地址、值、变量和分支守卫中被实际参数表达式替换, 并且调用点谓词应用于每个传播的操作。同一机制通过链接器/源限定的符号标识跨翻译单元运作。摘要展开是迭代的且有深度限制 (默认 3), 以防止通过互递归 helper 的无限递归。

这是一种轻量级的过程间分析: 我们不执行完整的上下文敏感摘要计算, 但有界深度的内联足以暴露隐藏在常见包装器模式 (如 gpio_setbit、reg_read 和 reg_write) 中的 MMIO 操作。

5.5 意图标注

每个操作都标注有一个意图字符串, 该字符串从解析的寄存器宏名称中派生。我们使用关键词匹配对寄存器名称进行分类: 包含 “INT” 或 “IRQ” 的寄存器被赋予意图 Interrupt; “CLK” 或 “PLL” 赋予 Clock; “CONFIG” 或 “CTRL” 赋予 Config; “STAT” 赋予 Status; 以此类推。这种轻量级启发式方法提供了人类可读的上下文, 无需深度语义分析。

语义推断

原始 RIS 捕捉了访问哪些寄存器以及如何访问, 但不解释为什么——即每个函数在驱动程序中扮演什么角色、它对设备状态有什么影响, 以及它必须满足什么契约。

REHARNES 通过三层推理为 RIS 模块丰富了后端无关的语义。

6.1 函数规约 (FunctionSpec)

FunctionSpec 是一个元组：

(Signature, Role, Context, Binds, Requires, Ensures, Effects, RISRef)

角色推断是关键步骤。REHARNES 优先使用回调表绑定而非函数名启发式。当回调表字段被识别时（例如 irq_chip 结构体中的 .irq_ack），相应的角色（interrupt_ack）和执行上下文（irq）被确定性地分配。该表覆盖了 irq_chip, platform_driver, virtio_config_ops, gpio_chip, dev_pm_ops, usb_ep_ops, usb_gadget_ops 和 hc_driver 中的标准 Linux 字段。对于未出现在任何回调表中的函数，后备启发式将函数名子串匹配到角色（例如 *_ack_irq → interrupt_ack）。

签名抽象将 C 参数类型映射到抽象形式类型：struct irq_data * → LogicalIRQ, struct platform_device * → DeviceState, 整数类型 → UInt, void → Void。这种抽象对于后端无关性至关重要：同一个 DeviceSpec 可以面向 Linux、裸金属和 harness 后端，而无需引用 Linux 特定类型。

状态绑定识别函数使用的 MMIO 基址表达式（例如 g->base），并将其绑定到形式化的 dev.base 路径。该绑定通过扫描函数的 RIS 操作的地址表达式中的成员访问模式来发现。

效果从两个来源派生：(1) 对于函数写入的每个符号寄存器，发出 writes_register(REG) 效果；(2) 从函数的角色派生语义事件效果（例如 interrupt_ack → clears_interrupt(line)）。

Requires/Ensures 是从角色语义派生的 Hoare 风格的前置条件和后置条件。例如，interrupt_ack 函数不需要前置条件，并确保 interrupt_pending[line] == false；probe 函数需要 resources_available，并确保 device_state == READY。

6.2 设备规约 (DeviceSpec)

一个 DeviceSpec 将多个 FunctionSpec 实例组织为设备级模型：

(State, Resources, Registers, Functions, Invariants, Class)

设备类从驱动程序名称推断（例如 gpio-ftgpio010 → gpio_controller, virtio_mmio → virtio_mmio）。

状态从函数的需求推断：每个设备获得一个 MmioBase 状态字段；仅当源码声明或获取了 struct clk 时才添加消费端 Clock 字段（具有 clk_hw/clk_ops 的时钟提供者不会创建虚假的消费者资源）；中断相关角色添加 num_irqs: UInt。

资源从状态字段派生：绑定到 base 的 MmioResource，以及根据需要的 ClockResource 和 IrqResource。

寄存器从 RIS 的 register_map 中收集，其中包含驱动程序实际访问的每个符号寄存器（不是硬件定义的每个寄存器，而是驱动程序涉及的寄存器）。

不变式作为最小安全属性生成：对于支持中断的设备，断言不变式 forall line: UInt. line < num_irqs → valid_interrupt_line(line)。

6.3 后端绑定 (.bind)

一个 BindSpec 将 DeviceSpec 中的抽象概念绑定到具体的后端 API：

- **类型映射**：DeviceState → "struct ftgpio_gpio", MmioBase → "void __iomem *(Linux)或 uintptr_t (裸金属)"。
- **回调映射**：irq_chip.irq_ack = ftgpio_gpio_ack_irq。
- **原语映射**：MmioRead(B4) → "readl" (Linux) 或 mmio_read32 (裸金属)。
- **状态映射**：dev.base → "g->base"。
- **导出映射**：interrupt_ack as "ftgpio_ack_irq" (裸金属)。

默认绑定按后端生成，但可以自定义以支持非标准 API 或命名约定。

6.4 源码事实 (.facts)

对于 LLM 辅助综合，REHARNES 提取源码事实——即有助于重建但不应污染后端无关形式规约的信息：

- **头文件**：<linux/gpio/driver.h>, <linux/platform_device.h>
- **结构体定义**：ftgpio_gpio, 字段为 base: void __iomem *, clk: struct clk *。
- **常量**：具有整数值的非寄存器宏（标志位、状态码、位掩码）。
- **回调表**：irq_chip.irq_ack = ftgpio_gpio_ack_irq 等。
- **资源**：获取调用(devm_platform_ioremap_resource)及其绑定目标。
- **错误路径**：源码中的返回码（-ENOMEM, -ENODEV, PTR_ERR）。
- **Helper 调用**：重要的子系统函数(devm_gpiochip_add_data, bgpio_init)。

7 代码生成

REHARNES 包含一个多后端代码生成器，消费 (RIS, DeviceSpec, BindSpec) 来生成可编译的驱动程序代码。后端是插件：每个后端占据 src/generator/ 下的一个目录，包含一个 Python 模块（声明后端名称、输出语言和确定性绑定构造）和一个 prompt.md 翻译规则模板。注册表在导入时自动发现插件，因此添加目标平台只需新建一个目录——无需修改流水线。提供四个后端：用户态 harness C、裸金属 C、Linux 内核模块和 Rust no_std。代码发射通过统一桥接委托给 LLM；正确性由独立验证门控强制执行，而非信任模型。

7.1 用户态 Harness 后端

harness 后端生成一个自包含的 C 程序, 通过字节可寻址的固定大小内存缓冲区模拟 MMIO。原生和大端序的 8、16 和 32 位读写通过包装函数路由, 每个访问都记录一条追踪行:

```
[trace 0] W 0x20 = 0x00000000 ; GPIO_INT_EN
[trace 1] W 0x30 = 0xffffffff ;
        GPIO_INT_CLR
```

harness 包含设备状态结构体、从寄存器映射派生的寄存器常量、基于 RIS 的函数体 (使用后端绑定的原语和状态映射) 以及单元测试脚手架。整个 harness 使用标准 C 编译器 (cc -Wall) 编译, 生成可执行和可追踪的二进制文件。

7.2 裸金属后端

裸金属后端生成面向裸机环境的可移植 C 代码。它不使用 Linux 内核类型, 而是使用标准整数类型 (uintptr_t, uint32_t) 和可移植的 MMIO 原语 (mmio_read32, mmio_write32)。生成的代码包括设备状态结构体、init/reset/IRQ 辅助函数, 不含框架粘合代码。使用 cc -ffreestanding -c 编译。

7.3 Linux 后端

Linux 后端生成可构建的内核模块, 包含 struct device_state、probe/remove 函数、框架操作表以及使用标准 Linux MMIO 原语 (readl, writel) 的基于 RIS 的回调函数体。它为平台 MMIO 设备 (devres 映射、可选消费端时钟、GPIO/IRQ 注册、时钟提供者注册和 OF 匹配)、PCI MMIO 设备 (enable/request/map/unmap/release 和 PCI ID) 以及 QEMU edu (暴露 MMIO 预言机的 miscdevice open/read/write 操作) 提供确定性的资源生命周期。对于保守识别的时钟提供者, 在显式私有容器重绑定后, 从版本化源码中保留不同的 clk_ops 实例和非 MMIO 速率运算。类似保守的源码模型恢复 Sodaville 使用的原生字节序 32 位 dat/set/dirout GPIO 模式和通用芯片 mask/unmask/EOI 契约; 不支持的变体被拒绝而非近似处理。不支持的源码私有状态仅被中和以保持结果可构建, 并伴随一个 REHARNES_UNSUPPORTED 标记, 阻止严格就绪声明。

7.4 Rust 裸金属后端

Rust 后端生成使用 tock-registers crate 进行类型安全 MMIO 的 no_std 驱动程序。提取到 RISregister_map 中的寄存器映射成为具有正确偏移量和保留填充的 register_structs! 声明; 每个寄存器的访问模式 (ReadOnly, WriteOnly, ReadWrite) 从观察到的操作类型派生。每个 RIS 模块成为驱动结构体上的安全方法; unsafe 仅限于转换基址的构造函数。该后端证明证据契约是语言无关的: 它与 C 后端共享相同的证据 JSON, 仅提示模板不同。

7.5 LLM 发射与验证门控

全部四个后端通过统一的 LLM 桥发射代码 (图 ??)。桥接将形式化产物简化为证据 JSON——驱动程序标识、设备类、寄存器映射、每模块操作列表 (包括事务和功能状态操作)、原语/类型/状态/回调绑定以及选定的源码事实——并用其实例化后端的提示模板。提示编码了目标平台的翻译规则 (例如, Linux 提示指定 PCI 与平台检测、miscdevice 连线和追踪插桩; Rust 提示指定 tock-registers 惯用法和禁止裸指针规则)。LLM 永远不会看到原始 C 源码, 因此发射被约束在提取的语义范围内。

因为 LLM 输出不可信, 每个候选都通过独立的验证循环:

1. **编译:** 候选代码使用目标后端的严格警告进行编译。
2. **静态检查:** 每个 RIS 引用解析到模块; 每个符号寄存器在寄存器映射中; 每个回调条目有表绑定; 没有遗留的 TODO。
3. **追踪检查 (仅 harness):** 生成的 harness 被执行, 其 MMIO 追踪与从 RIS 规范派生的预期追踪进行比较。
4. **修复:** 如果任何检查失败, 格式化结构化反馈连同当前候选发送给 LLM。LLM 被指示生成满足所有约束的修补候选。
5. **重复:** 步骤 1-4 迭代直到候选被接受或迭代预算 (默认 3) 耗尽。

候选只有在额外满足生成契约时才被接受: 每个发出的寄存器操作必须携带与其 RISop_id 和规范摘要关联的降级回执, 由独立的 AST 预言机精确检查一次, 拒绝缺失、重复、未知、类型不匹配和摘要漂移的操作。LLM 端点是通过 Pi agent SDK 实现的可插拔组件, 具有项目级配置。下面报告的冻结提取/编译矩阵和两个 QEMU 实验是在确定性发射器基线下产生的, 与模型选择无关; LLM 发射的输出通过验证门控单独评估, 因为模型和端点的不确定性否则会混淆产物复现。

8 评估

我们在 19 个版本化 Linux 驱动源码上评估 REHARNES, 覆盖 GPIO、时钟/PLL、AHCI、SDHCI、virtio-MMIO、帧缓冲光栅引擎和 QEMU edu。评估回答五个研究问题:

- **RQ1 (精确性):** 有多少比例的寄存器访问被解析为符号地址?
- **RQ2 (完备性):** 检测到多少 RMW 模式和分支条件?
- **RQ3 (生成):** 生成的 harness、裸金属和 Linux 目标能否编译?
- **RQ4 (对比):** REHARNES 与正则基线相比如何?
- **RQ5 (运行时):** 确定性生成的驱动是否在真实 QEMU 设备上保持寄存器行为?

8.1 实验设置

所有实验在配备 AMD 9950X 处理器和 64 GB 内存的 Linux 机器上运行, 使用 Python 3、libclang 18, 实验记录

Table 1. 19 个测试驱动的提取统计。Sym、Fixed 和 Comp 是不同的地址类别。

Driver	Ops	Sym	Fixed	Comp	RMW	Cond	Regs	%Sym
gpio-ftgpio010	35	35	0	0	7	8	13	100.0%
virtio_mmio	59	46	12	1	0	44	31	78.0%
gpio-pl061	31	27	0	4	7	1	7	87.1%
gpio-cadence	32	32	0	0	4	8	12	100.0%
edu	5	3	2	0	0	2	3	60.0%
Total (19)	485	366	64	41	73	123	157	77.7%

Table 2. 真实多源 Linux 驱动模块。X-TU 是已解析的跨翻译单元调用边数，Prop 是携带 MMIO 摘要的调用边数。H/B/L 报告生成的后端编译情况。† 表示在仅警告的 modpost 重试后成功的原始 Kbuild 编译/链接，因为固定的实验内核省略了 USB 核心导出。

Driver module	TUs	LoC	Funcs	X-TU	Prop	Src MMIO	RIS MMIO	Ops	H/B/L	Orig. Kbuild
aspeed-vhub	5	3540	92	53	54	101	154	158	✓/✓/✓	✓†
c67x00	4	2239	89	30	79	2	32	38	✓/✓/✓	✓†
dwc2	10	21668	445	140	445	804	3608	4250	✓/✓/✓	✓†

在 research/experiments/results/matrix.json 中。内核配置（包括 CONFIG_COMMON_CLK=y）、QEMU 脚本、驱动语料库和源码哈希随工件一同版本化管理。完整的 19 个驱动提取/生成/Kbuild 矩阵在实验内核构建完成后大约需要 108.0 秒（平均每个驱动 5.69 秒）。

8.2 提取覆盖率 (RQ1, RQ2)

表 ?? 直接从机器可读实验结果生成，总结了所有 19 个驱动。

关键发现：REHARNESS 保留三种地址类别，而非强制将所有访问归入符号名称类别。在含有地址的操作中，77.7% 为符号地址，而固定偏移和运行时计算的地址保持显式。这纠正了早期评估中将“已解析”与“符号化”混淆的错误。这一区分至关重要：索引化的 virtio 配置访问和分组 GPIO 寄存器可以被表示，但无法诚实地赋予一个静态寄存器名。

检测到的 73 个 RMW 模式集中在中断掩码/取消掩码回调和配置函数中。直线变异如 val &= B0T(line) 作为变换保留。分支依赖的 switch 更新表示为嵌套的 Ite 表达式，每个分支独立应用于原始读取值，而非错误地拼接。

记录的 123 个分支条件主要出现在配置和 probe 函数中，这些函数在修改寄存器之前检查设备状态。

8.3 多源驱动扩展

初始语料将一个 C 翻译单元视为一个提取目标。为了评估真实模块规模，我们为由四个或更多 C 文件组成的 Kbuild 模块添加了版本化清单。每个清单都根据所固定 Linux 模块的实际 *y 对象列表进行校验；这些文件并非事后归组的无关驱动。提取器为每个翻译单元构建独立的预处理上下文，合并宏和源码事实，并在构建一个组合的 RIS 和 DeviceSpec 之前，迭代地跨文件边界传播参数实例化的 MMIO 摘要。实验还从源码目录的临时副本编译原始模块，并将词法源码 MMIO 原语与直接和传播后的 RIS 操作进行比较。

Table 3. 每个驱动的生成就绪度评分。

Driver	RIS	FuncSpec	DevSpec	H/B/L compile	Linux ready	LLM ready
gpio-ftgpio010	1.000	1.000	1.000	✓/✓/✓	✓	✓
virtio_mmio	0.897	0.875	1.000	✓/✓/✓	-	-
gpio-pl061	0.924	1.000	0.938	✓/✓/✓	-	✓
gpio-cadence	1.000	1.000	1.000	✓/✓/✓	-	✓
edu	0.920	1.000	0.688	✓/✓/✓	-	✓

表 ?? 覆盖 3 个模块、19 个翻译单元、27447 行源码和 626 个函数，产生 4446 个操作、959 个 RMW 模式和 68 个映射寄存器。在 974 条内部调用边中，所有 223 条跨翻译单元边均已解析；578 条边携带 MMIO 摘要。源码比较发现 907 个词法 MMIO 原语和传播后的 3794 个 RIS MMIO 操作。C67X00 演示了其通过其底层 HPI 实现的传播，ASPEED vHub 覆盖了端点/集线器路径，而十文件的 DWC2 案例则涵盖了主机、外设、双角色、端点和 DMA 代码。三个模块的全部三个生成后端均可编译。C67X00 现在将 HPI 基址、寄存器步长和 SIE 号绑定为源码私有状态，并降低了全部 32 个计算的 HPI 地址。ASPEED vHub 和 DWC2 保留了显式的端点/HCD 生命周期和源码私有状态阻塞项。编译与严格就绪度仍然是不同的：C67X00 仍然记录了一个 switch 排他性缺口、一个未证明的循环以及不支持的外部 HCD 生命周期语义。

8.4 生成就绪度 (RQ3)

表 ?? 将编译与严格语义就绪度区分开来。一个生成的文件可以在编译通过的同时仍然携带一个显式的不支持状态绑定标记。

harness 输出在 19/19 个驱动上编译，裸金属在 19/19 个驱动上编译，Linux 在 18/19 个驱动上编译。相比之下，严格就绪度方面 harness 为 4/19，裸金属为 4/19，Linux 为 2/19；5/19 个驱动具有足够的结构化上下文以供辅助合成。从 gpio_generic_chip_config 恢复的通用 GPIO 回调由用户态 harness 中的契约驱动运行器和宏门控的裸金属主机预言机模式执行。一个独立的 Formal-RIS 解释器检查回调标识、操作类型、偏移、字节序和值；只有当每个合成的回调都通过时才恢复就绪度。Highbank 另行展示了源码感知的仅 Linux 时钟降阶。未知的变换、不安全的计算地址、目标-源码 clang 错误、不支持的子系统域以及规范化的源码私有状态即使在代码可以编译时也会阻塞就绪度。平均 RIS 质量为 0.902。

一个冻结的 12 驱动零样本留出集提供了独立的泛化性检查。三个生成后端在 12/12 个案例中均可编译，而严格就绪度在全部三个后端上对 5/12 个案例成立。顺序 gpio-mmio 回调通过与具有 sdata/sdir 影子状态的独立源码模型进行校验。SDHCI NPCM、Dove 和 HLWD 通过了源码访问器契约，包括 HLWD 的字节交换地址、传输模式影子化和延迟语义。Virtio 配置和 virtqueue 操作被表示为子系统私有状态而非虚假的 MMIO。对于 DW APB，Linux 后端为每个固件 bank 分配一个 gpio_chip 和影子状态，将 IRQ 注册限制在源码已证明的 bank 中，并恢复父 IRQ 发现/域分发；PM、bank 寻址、生命周期和 IRQ 位/类型变异被独立检查。Altera IRQ 循环仅在显式的无新挂起位假设下作为掩码 W1C 排放来降阶。这

Table 4. REHARNESS 与正则基线在四个关键模式上的比较。

模式	正则基线	REHARNESS
宏定义寄存器偏移	0% 已解析	从 AST 恢复常量
包装器函数内联	MMIO 丢失	已内联 (深度 ≤ 3)
分支条件	始终为 None	记录 123 个条件
算术地址	不支持	符号/固定/计算地址保留
RMW 检测	不支持	73 个 RMW 模式

些契约仍限于已建模的寄存器和子系统行为，而非任意硬件等价。

我们还发布了 `research/experiments/results/reliability.json`。每个已识别的源码访问获得一个稳定的站点标识符，每个 RIS 叶节点携带源码证据和精度标签，路径谓词通过 Z3 检查可满足性和 switch 排他性。直接的 volatile 解引用、内联汇编、不支持的寄存器域和未建模的控制转移不会悄无声息地消失。报告将其严格判定显式限定于已识别的寄存器访问和控制模式。 `whole_program_complete` 现在是链接分析、所有翻译单元、内部调用、访问核算、显式 CFG、路径验证、switch 排他性、操作标识/证据、计算地址、值、循环和访问域的合取。其范围为 `manifest-internal`；它不对外部内核或子系统语义声明完备性。默认的单源评估将别名分析关闭，因此该值保持为 `false`。

我们另外在 `research/experiments/results/clock-model-boundary.json` 中记录了机器可读的时钟模型边界。生成的 Highbank 回调通过了一个独立 Python 参考的 22 个算术案例，同时 PLL divq、A9 总线移位和外设时钟递增公式的变异均被检测到。将相同的源码模型应用于 Visconti PLL 被拒绝，并给出显式的未绑定状态原因（PLL 基址、速率表/计数和锁）。这一负对照演示了一个保守的语法/所有权边界；Highbank 预言机验证了版本化的公式，但并非硬件等价证明。

对于 C67X00，`required-mode` 清单运行将每个翻译单元编译为 bitcode，链接全部四个单元，并调用一次 SVF WPA 传递。分析记录了可重现的链接 bitcode SHA-256 和源码/工具来源。链接运行未发现额外的普通指针别名，支持了困难部分在于 HPI 聚合状态而非别名发现的诊断。一个独立的预言机执行了五个源码原语和四个原始 C 与 RIS 的差分案例；寄存器索引、寄存器步长、包装器目标和操作顺序的变异均被检测到。

8.5 与正则基线对比 (RQ4)

我们将 REHARNESS 与 `driver-harness` 进行比较，后者是一个基于正则表达式的工具，曾作为 REHARNESS 的前身和基线。比较集中在第 ?? 节中识别的四种失败模式上。

正则基线解析了 0% 的宏定义寄存器偏移，因为它无法评估预处理常量。它遗漏了包装器函数内部的 MMIO 操作，对分支条件报告 None，并且无法分类算术地址表达式。REHARNESS 通过 AST 级分析解决这些模式，同时保留 366 个符号、64 个固定和 41 个计算地址操作之间的区分。因此它既避免了基线的访问丢失，也避免了每个可表示地址都有一个静态符号名的不可靠声明。

8.6 案例研究：gpio-ftgpio010

我们详细考察 FTGPIO010 GPIO 控制器驱动。目标文件直接定义了 7 个含 MMIO 的模块；通用 GPIO 库契约从类型化配置中贡献了 7 个额外的合成回调。该驱动定义了 13 个寄存器宏，现在全部被直接或库摘要操作引用。

REHARNESS 从 14 个模块中提取了 32 个操作：22 个直接操作和 10 个类型化库摘要操作。它解析了 13 个寄存器，检测到 10 个 RMW 模式，并记录了 2 个条件。直接回调保留了其外层结构绑定，而合成回调绑定到 `gpio_chip.get/set/get_multiple/set_multiple/direction` 字段。`set_irq_type` 中三个 switch 依赖的 RMW 操作保持为嵌套的 `Ite` 变换，没有 Top。

生成的用户态 harness 使用 `cc -Wall`，裸金属 C 使用 `cc -ffreestanding -c`，Linux 输出通过固定内核的 Kbuild 接口构建。在 QEMU 中，版本化的平台设备注册器触发生成驱动的 probe 路径，GPIO chardev 练习器调用通用回调。其插桩追踪与 RIS 派生的预言机匹配，模块覆盖率为 6/6，调用覆盖率为 7/7，操作覆盖率为 13/13，寄存器覆盖率为 8/8 (`TRACE_MATCH_OK`)。

8.7 确定性运行时验证 (RQ5)

运行时实验由 `./run.sh qemu-experiments` 运行并记录在 `research/experiments/results/qemu.json` 中；两个实验均不调用 LLM。对于 QEMU edu，确定性 PCI 后端生成一个 `miscdevice`，在映射的 BAR 上提供位置读写。客户机练习器在偏移 0x00 处验证设备 ID，在偏移 0x04 处验证取反的活体检查值，在偏移 0x08 处验证阶乘结果。所有值级检查均通过 (`EDU_TRACE_OK`)。

对于 `gpio-ftgpio010`，实验注册一个平台设备，加载插桩的确定性模块，并运行一个 GPIO chardev 练习器来触发通用芯片方向、`get-multiple` 和 `set-multiple` 回调。插桩输出精确的运行时函数边界，预言机将每个选定的调用段与结构化的 Formal RIS 操作匹配一次，防止一条全局追踪被归功于多个回调。预言机通过 (`TRACE_MATCH_OK`)，模块覆盖率为 6/6，调用覆盖率为 7/7，操作覆盖率为 13/13，寄存器覆盖率为 8/8。这些实验验证了两个互补的性质：建模 PCI 设备上的具体读/写值，以及 Linux 平台驱动 probe 和已执行的子系统回调中的访问顺序/偏移保持。

8.8 案例研究：含功能状态的多源 SPI (dw-apb-ssi)

为了综合评估混合 IR 增强提取、功能状态操作和 LLM 发射的四后端生成，我们将 REHARNESS 应用于 Synopsys DesignWare APB SSI SPI 控制器 (`snps,dw-apb-ssi`)，该驱动从一个多源清单中提取，清单覆盖了固定内核树的 SPI 核心和 MMIO 翻译单元。

该驱动恰好暴露了促使混合流水线设计的盲区。其寄存器访问器 (`dw_readl/dw_writel`) 和中断掩码辅助函数 (`dw_spi_mask_intr`) 是定义在头文件中的 `static inline` 函数；仅源文件的 AST 传递会遗漏通过它们路由的每一个访问。通过将头文件内联到提取缓存并按函数和偏移合并 IR 证据，提取的 RIS 覆盖了 28 个模块和

244 个带证据操作, 涵盖控制寄存器 (SSIENR、BAUDR、CTRLR0/1、SER、IMR) 和数据通路 (DR、TXFLR、RXFLR)。功能状态层通过发送和接收路径来验证: 提取的 dw_writer 模块保留 txw := VALUE*(u8*)(dws->tx)) → STATE(dws->tx) := dws->tx + dws->n_bytes → W(B4, DW_SPI_DR) = txw → STATE(dws->tx_len) := dws->tx_len - 1, 而 dw_reader 对称地保留通过 OUT*(u8*)(dws->rx)) := rxw 的读到缓冲区路径。宽度相关的 8/16/32 位缓冲区访问保持为不同的条件分支。

全部四个后端均由此单一 RIS 通过 LLM 桥生成: harness C、裸金属 C、Linux 内核模块和使用 tock-registers 的 Rust no_std 驱动。生成的输出通过了全部 18 项独立语义检查, 涵盖各后端编译、set_cs 中的片选断言、中断掩码/取消掩码序列、传输配置和上述有序的 tx/rx 缓冲区数据流。工件 (包括中间的 .ris、.dspec 和 .facts 文件) 版本化管理在 examples/dw-apb-ssi/ 下。

本案例研究的两个观察影响了设计。第一, IR 证据是补充而非替代: IR 传递可靠地恢复了头文件内联的 MMIO, 但宏名称、控制谓词和回调绑定仅存在于 AST 层面。第二, 寄存器级等价对于数据搬运驱动是不够的: 一个早期迭代删除了 STATE/VALUE/OUT 操作, 生成的后端以正确顺序触及了正确的寄存器, 却无法传输一个字节, 因为缓冲区游标从未前进。

9 相关工作

9.1 驱动分析与提取

C 到 Rust 翻译。 C2Rust 项目 [?] 执行从 C 到 unsafe Rust 的语法翻译, 将安全所有权和借用标注的插入留给后续工具。&inator [?] 通过将 Rust 类型选择建模为类型片段、指针变换图和 NLL 借用约束上的 SMT 约束满足问题来解决接口翻译问题。REHARNESS 是互补的: 它提取形式化的寄存器级行为, 可以指导 C2Rust 式翻译器中的语义保持约束。

静态驱动分析。 SDV [?], Dingo [?] 和 Termite [?] 对驱动源码执行静态分析以验证协议合规性并生成样板代码。这些工具运作在框架 API 级别 (例如, 验证 probe 分配资源且 remove 释放资源)。REHARNESS 运作在更低层: 它提取实际的寄存器交互序列, 捕捉的是硬件协议而非操作系统框架协议。

符号执行。 KLEE [?] 和 S2E [?] 对驱动代码执行符号执行以生成测试输入。REHARNESS 的提取是静态而非动态的, 使其适用于无法在测试环境中执行的驱动 (例如, 由于缺少硬件或内核基础设施)。

9.2 形式化规约与代码生成

设备驱动综合。 Termite-2 [?] 从设备和操作系统接口的形式化规约合成驱动。规约必须用领域特定语言手动编写。REHARNESS 自动化了规约提取步骤, 从现有 C 源码自动生成 DeviceSpec 和 BindSpec。

基于 LLM 的代码生成。 近期工作探索了使用大语言模型进行驱动综合 [?]。这些方法通常基于自然语言描述或源码片段运作。REHARNESS 的 LLM 辅助综合模块的

独特之处在于, 它提供结构化的形式化规约和验证反馈作为输入约束, 实现修复循环而非一次性生成。

9.3 驱动的中间表示

设备树和 ACPI。 设备树和 ACPI 等硬件描述标准描述硬件拓扑 (寄存器基址、中断线、时钟), 但不描述寄存器交互协议。REHARNESS 的 RIS 和 DeviceSpec 通过捕捉行为协议来补充这些标准。

Haiku 和 Barrelfish 驱动模型。 Haiku [?] 和 Barrelfish [?] 操作系统使用将设备逻辑与操作系统框架代码分离的驱动模型, 类似于 REHARNESS 将 DeviceSpec (设备做什么) 与 BindSpec (如何连接到操作系统) 分离的方式。REHARNESS 提供了从现有 Linux 驱动中提取这些规约的工具。

10 讨论与局限

10.1 当前局限

Linux 子系统覆盖。 REHARNESS 识别 GPIO 和 IRQ 表 (gpio_chip、irq_chip 和 gpio_irq_chip); 总线驱动 (platform_driver、pci_driver 和 amba_driver); virtio、时钟、电源管理、文件操作、SPI 控制器和 USB 端点/外设/HCD 表。类型化事务识别通过来源门控覆盖公共 regmap、I2C/SMBus 和 MFD 辅助 API。USB 回调签名和常见私有字段已被建模, 但端点分配、外设注册和主机控制器生命周期重建仍然作为显式的不支持绑定。其余子系统需要额外的回调和生命周期模型。

数据流精度。 流敏感分析保留分支谓词, 并将常见的 switch/if RMW 模式合并为条件表达式。它构建显式的源码级基本块, 包含前驱/后继边、支配/后支配关系、汇合、goto 边、回边和循环头。简单的仅参数提前退出、有界的前向 goto、规范化的静态有界 for 循环以及经过专门验证的掩码 W1C 排放可以被降阶; 后向 goto 和未证明的循环仍然是显式阻塞项。W1C 界限依赖于一个记录的静止假设, 不覆盖并发中断到达。分析仍然不为任意路径、递归或外部框架状态维护通用的抽象存储不动点。SVF 别名分析可作为可选模式 (auto 或 required) 使用, 因为其外部工具链和分析成本否则会削弱确定性复现。多源清单使用链接的 bitcode 和一次 WPA 传递, 而非每个翻译单元单独 SVF; 默认评估使用别名模式 off。

Linux 后端完备性。 确定性 Linux 后端为 18/19 个驱动输出可 Kbuild 编译的结果, 并实现了具体的平台、PCI、GPIO/IRQ、时钟、电源管理和 edu 生命周期。编译不被视为语义完成: 仅 2/19 个严格就绪。常见的 GPIO、时钟、virtio、USB 控制器和端点私有字段被表示在生成的设备状态中。顺序 GPIO 影子状态是显式的, 但并发回调和 pinctrl 行为超出了当前预言机范围。GPIO IRQ 初始化和时钟提供者回调现在保留了具体的表绑定; 时钟速率回调仅在保守的私有状态重绑定后保留源码标量算术。Sodaville 的源码派生 GPIO/IRQ 生命周期由结构断言和 Kbuild 覆盖, 但没有可用的真实 Sodaville 硬件或等效设备模型。公共 SDHCI 访问器仅在源码结构排除私有覆盖时才被接受; 这不重建完整的主机生命周期。正确的 USB 回调签名和操作表条目被输出, 但不完整的 USB

核心生命周期绑定和其他子系统特定对象仅以显式的不支持标记进行规范化。未知的变换和不安全的计算地址同样阻塞严格就绪度。

计算地址。 REHARNESS 为索引化的寄存器 bank 保留完整的运行时偏移表达式，并在每个项均可降阶时直接输出。这样的访问仍然缺少一个静态符号寄存器名，但它本身并非不安全。包含未知调用、未绑定成员或 Top 的表达式仍然是显式的就绪度阻塞项。

LLM 可复现性。 当输出委托给 LLM 后端时，模型版本、端点和采样会为生成文本引入不确定性。REHARNESS 通过保持证据契约和所有验证门控的确定性来缓解这一问题：相同的证据 JSON 和提示产生候选，这些候选无论由哪个模型生成，都必须通过相同的编译、AST 接收和追踪检查。上述冻结的提取/编译矩阵和 QEMU 结果是在确定性发射器基线下产生的；LLM 发射的输出（如 dw-apb-ssi 四后端案例研究）及其独立验证结果另行报告。具有足够结构化上下文的候选（在我们的就绪度指标中为 5/19）有资格进行辅助合成。

功能等价范围。 功能状态操作将等价声明从寄存器级（相同寄存器、相同顺序、相同值）扩展到功能级（数据实际在缓冲区和设备之间搬运）。当前范围覆盖缓冲区游标、剩余长度计数器和直线及分支路径上的输出存储。完整的协议状态建模——跨中断边界的传输状态机、DMA 描述符生命周期和并发完成路径——仍然是未来工作；生成的 dw-apb-ssi 驱动保留了源码数据流顺序，但尚未针对真实 SPI 外设硬件进行验证。

时间复杂度。 libclang 解析步骤是主要开销：解析一个典型的 Linux 驱动（包含完整内核头文件）需要 10–30 秒。这对于单个驱动的批量分析是可接受的，但对全内核分析来说将是不可承受的。

10.2 未来工作

更多后端。 插件架构使新目标只需一个 prompt 加模块目录。Rust no_std 后端就是以这种方式添加的，无需修改流水线；我们计划为 RTOS 目标（FreeRTOS、Zephyr）和验证目标（CBMC、SeaHorn）添加后端，它们使用相同的 DeviceSpec 和证据契约，但使用不同的绑定和提示文件。

有状态 RIS 建模。 当前 RIS 捕捉操作序列但不建模设备状态转换。一个建模寄存器状态机（例如，寄存器值上的有限状态自动机）的有状态扩展将支持更强的验证，例如检查驱动从不读取未初始化的寄存器。

全内核分析。 将提取扩展到数千个内核驱动需要增量解析、宏表和调用图的缓存以及并行执行。REHARNESS 的模块化特性（按驱动、按函数提取）使这成为可能。

与安全语言翻译的集成。 自然的下一步是将 REHARNESS 与 C2Rust 式编译器集成：提取的 RIS 和 DeviceSpec 可以为类型选择和所有权标注决策提供信息，特别是对于 MMIO 指针和中断处理上下文。

11 结论

我们提出了 REHARNESS，一个使用 libclang AST 分析结合流敏感数据流与污点追踪从 C 设备驱动中提取形式

化寄存器交互规约的流水线，以 LLVM IR 证据补充恢复头文件内联的 MMIO 辅助函数。REHARNESS 克服了基于正则表达式提取的四个基本局限：宏定义的寄存器偏移通过预处理记录解析，包装器函数通过跨过程调用图分析内联，分支条件通过谓词堆叠记录，算术地址表达式通过符号抽象解释求值。RIS 语言还额外建模了类型化的 regmap/I2C/MFD 事务和保持缓冲区到寄存器数据流的功能状态操作。

在原始提取之上，REHARNESS 推断后端无关的函数和设备语义，并通过可插拔的四后端架构（harness C、裸金属 C、Linux 内核模块、Rust no_std）生成目标代码，其中提示驱动的 LLM 发射受独立编译、AST 接收和追踪验证门控。在 19 个驱动上，它提取 485 个操作——366 个符号、64 个固定和 41 个计算地址——包括 73 个 RMW 模式和 123 个条件，平均 RIS 质量评分为 0.902。在冻结的确定性基线下，harness 和裸金属输出在 19/19 个驱动上编译，Linux 输出在 18/19 个驱动上编译，而显式的 2/19 严格 Linux 就绪度结果防止将编译误认为语义完备性。值级和追踪级 QEMU 预言机对确定性 edu 和 gpio-ftgpio010 输出通过，LLM 发射的四后端 dw-apb-ssi 案例研究通过全部 18 项独立语义检查，包括功能等价所需的有序 tx/rx 缓冲区数据流。

REHARNESS 使得在寄存器级对驱动行为进行形式化推理成为可能，并通过功能状态建模开始将这种推理扩展到数据移动。通过以形式化语言而非临时格式生成规约——并将 LLM 输出视为需要验证的不可信候选而非可直接接受的结果——它为验证、测试、安全语言翻译和 AI 辅助驱动综合提供了基础。

References

- [1] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler. An empirical study of operating systems errors. In *Proc. SOSP*, 2001.
- [2] M. M. Swift, B. N. Bershad, and H. M. Levy. Improving the reliability of commodity operating systems. In *Proc. SOSP*, 2003.
- [3] T. Ball, E. Bounimova, B. Cook, V. Levin, J. Lichtenberg, C. McGarvey, B. Ondrusek, S. K. Rajamani, and A. Ustuner. Thorough static analysis of device drivers. In *Proc. EuroSys*, 2006.
- [4] L. Ryzhyk, P. Chubb, I. Kuz, and G. Heiser. Dingo: Taming device drivers. In *Proc. EuroSys*, 2009.
- [5] A. Kadav, M. J. Renzelmann, and M. M. Swift. Tolerating hardware device failures in software. In *Proc. SOSP*, 2009.
- [6] T. Witkowski, N. Blanc, D. Kroening, and G. Weissenbacher. Model checking concurrent Linux device drivers. In *Proc. ASE*, 2007.
- [7] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proc. OSDI*, 2008.
- [8] M. Emrath, P. V. D. Voort, and F. V. D. Lijn. C2Rust: Translating C to Rust. <https://c2rust.com/>, 2021.
- [9] J. Roelofs, A. Kadav, and A. Cidon. Large language models as driver synthesizers. In *Proc. HotOS*, 2023.
- [10] driver-harness: Regex-based driver register extraction. <https://github.com/...>, 2025.
- [11] CRUST-Bench: A benchmark suite for C-to-Rust translation. 2024.
- [12] L. Ryzhyk, P. Chubb, I. Kuz, E. L. Sueur, and G. Heiser. Automatic device driver synthesis with Termite. In *Proc. SOSP*, 2009.
- [13] L. Ryzhyk, A. Walker, J. Keys, A. Legg, A. Raghunath, M. Stumm, and M. Vij. User-guided device driver synthesis. In *Proc. OSDI*, 2014.

- [14] V. Chipounov, V. Kuznetsov, and G. Candea. S2E: A platform for in-vivo multi-path analysis of software systems. In *Proc. ASPLOS*, 2011.
- [15] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach, and A. Singhanian. The multikernel: A new OS architecture for scalable multicore systems. In *Proc. SOSP*, 2009.
- [16] Haiku operating system. <https://www.haiku-os.org/>.
- [17] &inator: Correct, precise C-to-Rust interface translation. In *Proc. PLDI*, 2026.

Temporary page!

ℒ_TEX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because ℒ_TEX now knows how many pages to expect for this document.